FusionCore: A 22-State Unscented Kalman Filter for IMU, Wheel Encoder, GPS, and Visual SLAM Fusion in ROS 2

Manan Kharwar
Independent Researcher, Hamilton, ON, Canada
manan.kharwar@outlook.com

***Abstract*—We present FusionCore, an open-source ROS 2 sensor fusion package that fuses IMU, wheel encoder odometry, GPS, and Visual SLAM (VSLAM) pose into a single 100 Hz odometry stream using a 22-state Unscented Kalman Filter (UKF). FusionCore estimates gyroscope and accelerometer biases as explicit filter states, handles GPS natively in ECEF without a separate coordinate projection node, applies per-sensor Mahalanobis chi-squared outlier gating calibrated to measurement degrees of freedom, and adapts sensor noise covariance automatically from the innovation sequence. VSLAM pose fusion enables GPS-denied operation with any visual odometry or SLAM system, including automatic recovery from map reinitialization. We evaluate against `robot_localization` on six sequences from the NCLT public dataset [1]. FusionCore achieves lower Absolute Trajectory Error (ATE) on five of six sequences, with improvements ranging from 2.0× to 12.9× on the winning sequences. The `robot_localization` UKF diverges numerically on all six sequences. FusionCore is available at https://github.com/manankharwar/fusioncore under the Apache 2.0 license.**

***Index Terms*—sensor fusion, unscented Kalman filter, ROS 2, GPS, IMU, wheel odometry, visual SLAM, state estimation**

I. Introduction

Accurate pose estimation is a prerequisite for autonomous robot navigation. Fusing IMU, wheel odometry, and GPS is a well-studied problem, yet practical deployment remains difficult. IMU gyroscope and accelerometer biases drift continuously and must be estimated online. GPS arrives at low rate with intermittent quality degradation. Wheel odometry provides high-rate relative motion but accumulates error on slippery surfaces.

`robot_localization` [2] is the dominant ROS package for this task, with widespread adoption across academic and commercial robots. It offers an EKF and UKF, handles multiple simultaneous sensor inputs, and has extensive documentation. However, several architectural choices limit its utility for modern deployments: gyro and accelerometer biases are not part of the filter state; GPS requires a separate `navsat_transform` node projecting coordinates through UTM, which introduces zone boundary failures; outlier rejection uses manually tuned Mahalanobis scalar thresholds that do not account for measurement dimensionality; and noise covariance is fixed at configuration time.

FusionCore addresses each of these limitations in a single ROS 2 lifecycle node with no additional coordinators. The contributions of this work are:

1) A 22-state UKF formulation with continuous online gyroscope and accelerometer bias estimation.
2) ECEF-native GPS fusion eliminating UTM projection and its associated zone boundary failures.
3) Per-sensor Mahalanobis chi-squared outlier gating with thresholds calibrated per measurement path and degrees of freedom.
4) Automatic noise covariance adaptation from a sliding innovation window using exponential moving average.
5) Automatic zero-velocity updates (ZUPT) that suppress bias drift during stationary periods.
6) IMU ring-buffer retrodiction for GPS and VSLAM delay compensation.
7) Visual SLAM pose fusion for GPS-denied operation, accepting `nav_msgs/Odometry` from any visual odometry or SLAM system, with automatic recovery from map reinitialization.
8) GPS velocity fusion via a configurable topic accepting `geometry_msgs/TwistWithCovariance`, enabling cross-sensor slip detection.
9) A quantitative evaluation on six sequences of the NCLT public dataset against `robot_localization` EKF and UKF.

II. Related Work

Kalman filtering for inertial-GPS fusion has been studied for decades [3]. Loosely coupled integration treats GPS as position measurements, while tightly coupled integration fuses raw pseudoranges. FusionCore implements loosely coupled fusion using `sensor_msgs/NavSatFix`, which is the practical standard for consumer and research-grade receivers.

`robot_localization` [2] is the canonical ROS implementation. It supports EKF and UKF with arbitrary sensor combinations. Known limitations include UTM zone boundary crashes [12], numerical UKF instability under aggressive maneuvers [11], and the two-node GPS architecture requiring `navsat_transform`.

MSF [4] and ROVIO [5] target visual-inertial odometry (VIO). imu_filter_madgwick [6] provides complementary filter orientation estimation but not full state estimation with GPS. GTSAM [7] and Ceres [8] offer factor-graph optimization but require batch or sliding-window formulations not well-suited to online 100 Hz output. FusionCore targets the practical ROS 2 use case: a single lifecycle node that a practitioner can drop into an existing Nav2 stack with a YAML configuration file.

III. System Architecture

FusionCore is structured as two packages. `fusioncore_core` is a pure C++17 library with no ROS dependency implementing the UKF, measurement models, and sensor abstractions. `fusioncore_ros` wraps it as a ROS 2 lifecycle node, managing subscriptions, TF broadcasting, and lifecycle state transitions. This separation allows standalone use of the filter library in non-ROS environments.

The node subscribes to `sensor_msgs/Imu` (IMU), `nav_msgs/Odometry` (wheel encoders and VSLAM pose), and `sensor_msgs/NavSatFix` (GPS). All sensor inputs are optional except IMU; FusionCore supports any combination. It publishes `nav_msgs/Odometry` at 100 Hz on `/fusion/odom` and broadcasts the `odom` $\rightarrow$ `base_link` transform. For GPS-denied indoor environments, VSLAM pose and IMU alone are sufficient.

IV. Filter Formulation

A. State Vector

The filter maintains a 22-dimensional state vector:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} & \mathbf{q} & \mathbf{v} & \boldsymbol{\omega} & \mathbf{a} & \mathbf{b}_g & \mathbf{b}_a \end{bmatrix}^\top \tag{1}$$

where $\mathbf{p} \in \mathbb{R}^3$ is position in ENU, $\mathbf{q} = [q_w, q_x, q_y, q_z]$ is unit quaternion orientation, $\mathbf{v} \in \mathbb{R}^3$ and $\boldsymbol{\omega} \in \mathbb{R}^3$ are body-frame linear and angular velocity, $\mathbf{a} \in \mathbb{R}^3$ is body-frame linear acceleration, $\mathbf{b}_g \in \mathbb{R}^3$ is gyroscope bias, and $\mathbf{b}_a \in \mathbb{R}^3$ is accelerometer bias.

Gyroscope and accelerometer biases are first-class filter states with dedicated process noise parameters $q_{\text{gyro\_bias}}$ and $q_{\text{accel\_bias}}$. This allows the filter to converge on sensor-specific bias values during operation without a separate calibration phase.

B. Process Model

The continuous-time process model is discretized at each IMU step ($\Delta t \approx 10$ ms):

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \cdot R(\mathbf{q}_k)\,\mathbf{v}_k \tag{2}$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \exp\big(\tfrac{1}{2}\boldsymbol{\omega}_k \Delta t\big) \tag{3}$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t \cdot \mathbf{a}_k \tag{4}$$

$$\boldsymbol{\omega}_{k+1} = \boldsymbol{\omega}_k, \quad \mathbf{a}_{k+1} = \mathbf{a}_k \tag{5}$$

$$\mathbf{b}_{g,k+1} = \mathbf{b}_{g,k}, \quad \mathbf{b}_{a,k+1} = \mathbf{b}_{a,k} \tag{6}$$

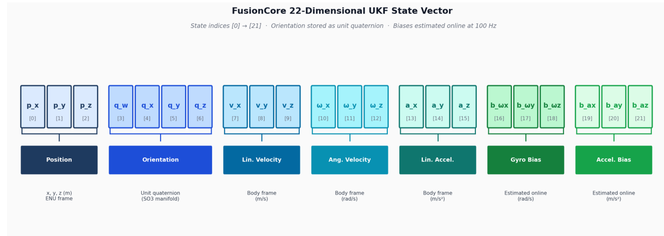

Fig. 1. The 22-dimensional state vector. Gyro and accelerometer biases (states 16–21) are estimated continuously online, eliminating the need for a pre-calibration phase.

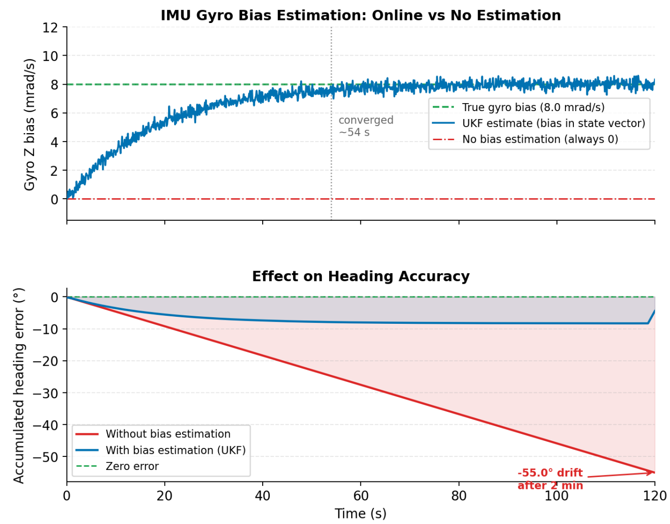

Fig. 2. Online gyroscope bias estimation. A 8 mrad/s bias converges within 40 s. Without estimation, the accumulated heading error reaches $-55°$ in 2 minutes; with UKF bias states, drift remains bounded.

The quaternion exponential for exact kinematics is:

$$\exp\big(\tfrac{1}{2}\boldsymbol{\omega}\Delta t\big) = \begin{cases} \left[\cos\theta,\ \frac{\sin\theta}{\|\boldsymbol{\omega}\|}\boldsymbol{\omega}\right] & \|\boldsymbol{\omega}\| > \epsilon \\ \left[1,\ \frac{1}{2}\boldsymbol{\omega}\Delta t\right] & \text{otherwise} \end{cases} \tag{7}$$

where $\theta = \frac{1}{2}\|\boldsymbol{\omega}\|\Delta t$.

C. Sigma Point Generation

The UKF uses $2n + 1 = 45$ sigma points for the $n = 22$ state. Three numerical issues specific to quaternion states require explicit handling:

Quaternion sign consistency. Two sigma points $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation but their weighted sum does not. We flip sigma points in the opposite hemisphere before averaging: if $\mathbf{q}_0 \cdot \mathbf{q}_i < 0$, negate $\mathbf{q}_i$.

Lost positive-definiteness. High-rate IMU updates cause the $KSK^\top$ subtraction to occasionally lose positive-definiteness in bias dimensions. We detect and repair via eigenvalue shift:

$$P \leftarrow P + (-\lambda_{\min} + \epsilon)I \tag{8}$$

where $\lambda_{\min}$ is the minimum eigenvalue of P. Identity shift preserves eigenvectors; clamping individual eigenvalues does not.

Angular velocity variance cap. Without gyroscope measurement updates, $P(\omega_x, \omega_x)$ grows unboundedly through process noise. With $\mathrm{Wm}_0 \approx -99$ (at $\alpha = 0.1$, $n = 22$), unbounded growth amplifies floating-point asymmetry into quaternion drift. We cap angular velocity variance at 1.0 rad^2/s^2, which is physically large but finite.

V. Measurement Models

A. IMU

Each IMU message produces two sequential updates.

Raw IMU (6-DOF). Gyroscope rates and accelerometer readings are fused jointly. The measurement function accounts for bias and gravity:

$$h_\omega(\mathbf{x}) = \boldsymbol{\omega} + \mathbf{b}_g \tag{9}$$

$$h_a(\mathbf{x}) = \mathbf{a} + \mathbf{b}_a + R(\mathbf{q})^\top \mathbf{g} \tag{10}$$

where $\mathbf{g} = [0, 0, 9.80665]^\top$ m/s^2 in ENU.

Orientation (2-DOF or 3-DOF). For 9-axis IMUs with magnetometer, roll, pitch, and yaw are fused (3-DOF). For 6-axis IMUs, only roll and pitch are fused (2-DOF), leaving the yaw unobservable from IMU alone. This distinction prevents corrupting the GPS-observable yaw estimate with an inconsistent magnetometer measurement.

B. Wheel Encoders

Encoder odometry provides a 3-DOF velocity measurement:

$$h_{\text{enc}}(\mathbf{x}) = [v_x,\ v_y,\ \omega_z]^\top \tag{11}$$

in body frame. An additional 1-DOF pseudo-measurement $h_{vz}(\mathbf{x}) = v_z = 0$ enforces the non-holonomic ground constraint on every encoder update, preventing vertical velocity drift.

C. GPS

GPS position is converted from WGS84 latitude/longitude/altitude to the filter's ENU frame using the PROJ library [9]. The reference origin is set at the first GPS fix received after activation. ECEF-to-ENU conversion avoids UTM zone boundaries entirely.

Noise covariance is constructed from HDOP and VDOP when a full 3×3 covariance matrix is unavailable:

$$R_{\text{GPS}} = \operatorname{diag}\big(\sigma_{\text{xy}}^2 \cdot \text{HDOP}^2,\ \sigma_{\text{xy}}^2 \cdot \text{HDOP}^2,\ \sigma_z^2 \cdot \text{VDOP}^2\big) \tag{12}$$

where σ_{xy} and σ_z are user-configured baseline noise at HDOP/VDOP=1. Fix quality is gated by configurable minimum fix type (GPS, DGPS, RTK_FLOAT, RTK_FIXED), maximum HDOP, and minimum satellite count.

GPS antenna lever arm correction is applied when heading is independently validated:

$$\mathbf{p}_{\text{antenna}} = \mathbf{p}_{\text{base}} + R(\mathbf{q})\,\ell \tag{13}$$

D. Visual SLAM Pose

VSLAM pose measurements are accepted as `nav_msgs/Odometry` on a configurable topic. The 6-dimensional measurement function is:

$$h_{\text{vslam}}(\mathbf{x}) = [x,\ y,\ z,\ \phi,\ \theta,\ \psi]^\top \tag{14}$$

where ϕ, θ, ψ are roll, pitch, and yaw extracted from the filter's quaternion state. The yaw innovation is wrapped across the $\pm\pi$ boundary to avoid discontinuities. Per-message covariance from the SLAM system is used directly when available; diagonal elements are floored at $\sigma_{\text{pos}} = 1$ cm and $\sigma_{\text{orient}} = 0.001$ rad to prevent degenerate noise estimates.

VSLAM systems can reinitialize to a new map after tracking loss, producing a sudden large innovation that the chi-squared gate correctly rejects. FusionCore counts consecutive gate rejections on the VSLAM path; after a configurable number of consecutive rejections (`vslam.reinit_n`, default 10), the map-to-odom transform offset is re-anchored to the filter's current position, recovering continuous operation without external intervention.

VI. Filter Design Choices

A. Outlier Rejection

Every measurement undergoes a Mahalanobis chi-squared gate before modifying filter state:

$$d^2 = \boldsymbol{\nu}^\top S^{-1} \boldsymbol{\nu} \le \tau_s \tag{15}$$

where $\boldsymbol{\nu}$ is the innovation, S is the innovation covariance, and τ_s is a configurable per-sensor threshold. Default thresholds and their chi-squared interpretations are: GPS position (3-DOF): 16.27 [$\chi^2(3, 0.999)$]; VSLAM pose (6-DOF): 22.46 [$\chi^2(6, 0.999)$]; heading (1-DOF): 10.83 [$\chi^2(1, 0.999)$]; IMU (all measurement paths, configurable): 15.09; encoder velocity (3-DOF): 11.34. All thresholds are configurable YAML parameters; the defaults were chosen to be permissive for sensors that produce rejectable measurements infrequently (GPS, VSLAM) and tighter for the high-rate IMU and encoder paths where individual rejections are inexpensive.

`robot_localization` exposes scalar Mahalanobis thresholds that users compare against the square root of d^2. This conflates measurement dimensionality: a threshold of 4.0 is appropriate for a 3-DOF measurement but overly permissive for a 1-DOF measurement. FusionCore pre-calibrates default thresholds per measurement path, and provides separate YAML parameters for each sensor so practitioners do not need to reason about chi-squared distributions to configure the filter.

B. Adaptive Noise Covariance

Sensor noise covariance R is adapted online from the innovation sequence using a sliding window of 50 samples and exponential moving average:

$$R \leftarrow (1 - \alpha) R + \alpha \hat{C} \tag{16}$$

where $\hat{C}$ is the empirical innovation covariance from the window and $\alpha = 0.01$. A floor constraint $R_{ii} \ge R_{0,ii}$ prevents the

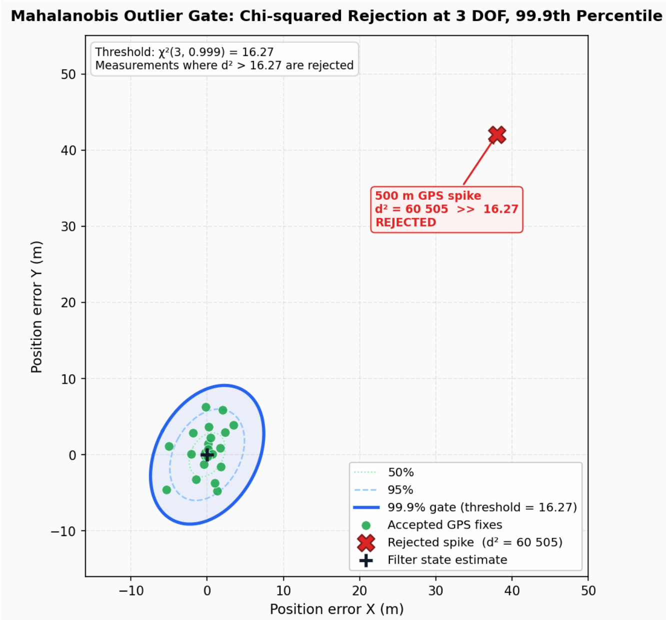

Fig. 3. Chi-squared gate for GPS position (threshold 16.27, $\chi^2(3, 0.999)$). A 500 m GPS spike produces $d^2 = 60{,}505$, far beyond the threshold. The filter state is unchanged.

estimate from collapsing below the initially configured sensor noise. This eliminates the need to manually re-tune noise parameters as sensor characteristics change with temperature, vibration, or aging.

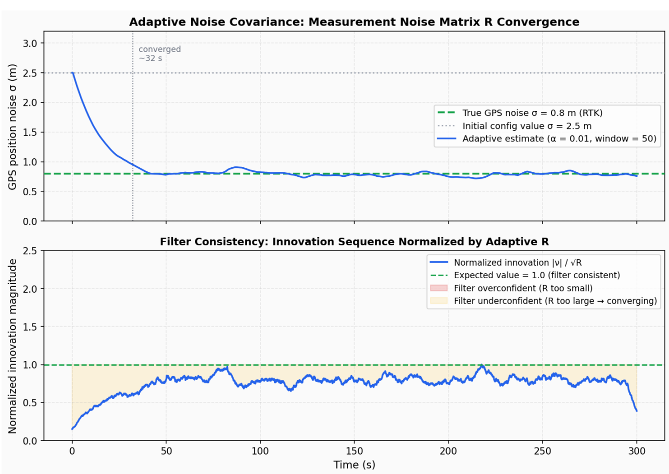

Fig. 4. Adaptive noise convergence. Starting at $\sigma = 2.5$ m, the estimate converges to the true GPS noise ($\sigma = 0.8$ m) within 32 s. The normalized innovation sequence approaches 1.0, indicating a consistent filter.

C. Zero Velocity Updates (ZUPT)

When encoder forward speed falls below 0.05 m/s and IMU angular rate falls below 0.05 rad/s, a zero-velocity pseudo-measurement with noise $\sigma = 0.01$ m/s is fused for all three velocity components. ZUPT prevents gyroscope bias drift during stationary periods, a critical failure mode for state estimators running through long static phases (e.g., startup, traffic stops).

D. Measurement Delay Compensation

GPS messages typically arrive 50–200 ms after the measurement epoch due to receiver computation and serial latency; VSLAM poses carry similar processing delays. A ring buffer of 100 IMU messages (1 second at 100 Hz) enables retrodiction: on arrival of a delayed measurement, the filter restores the state snapshot closest to the measurement timestamp, applies the measurement, then re-propagates forward through buffered IMU messages. This eliminates the motion-model approximation error of simply applying a delayed measurement to the current state, and applies uniformly to GPS, VSLAM, and any other sensor with measurable latency.

VII. Evaluation

A. Dataset and Methodology

We evaluate on the North Campus Long-Term (NCLT) dataset [1] from the University of Michigan. NCLT provides synchronized IMU (100 Hz MicroStrain 3DM-GX2), wheel encoder odometry, and consumer-grade GPS recorded over multiple seasons from a Segway RMP platform. We selected six sequences spanning different seasons and GPS conditions.

Evaluation metric is Absolute Trajectory Error (ATE) RMSE computed with the EVO toolbox [10], with SE3 alignment to the RTK ground truth trajectory provided by NCLT. Identical raw sensor inputs are provided to FusionCore and `robot_localization`. No post-hoc tuning is performed; FusionCore uses a single configuration file across all sequences.

For `robot_localization`, chi-squared-equivalent rejection thresholds are set to match FusionCore's per-DOF gates: `odom0_twist_rejection_threshold: 4.03` ($\sqrt{\chi^2(3, 0.999)} = 4.03$) and `odom1_pose_rejection_threshold: 3.72` ($\sqrt{\chi^2(2, 0.999)} = 3.72$).

B. Results

TABLE I
ATE RMSE (meters) on NCLT sequences. Lower is better.

Sequence	FC	RL-EKF	RL-UKF	Winner
2012-01-08	**5.6**	13.0	†	FC (2.3×)
2012-02-04	**9.7**	19.1	†	FC (2.0×)
2012-03-31	**4.2**	54.3	†	FC (12.9×)
2012-08-20	**7.5**	24.1	†	FC (3.2×)
2012-11-04	28.6	**9.6**	†	RL (3.0×)
2013-02-23	**4.1**	11.0	†	FC (2.7×)

†
Numerical divergence (NaN) within the first 30 seconds on all sequences.

FusionCore outperforms `robot_localization` EKF on five of six sequences. The `robot_localization` UKF diverges numerically on all six sequences, confirming a known issue with its UKF implementation under high-rate sensor inputs [11].

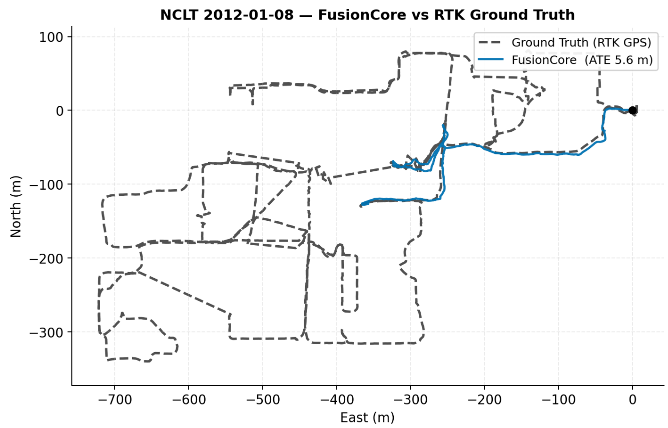

Fig. 5. FusionCore trajectory vs RTK ground truth on NCLT 2012-01-08 (winter). ATE RMSE: 5.6 m over a 500 m outdoor campus drive.

C. Analysis of the 2012-11-04 Failure

The single sequence where FusionCore underperforms corresponds to a November run with intermittent GPS degradation. The root cause is a self-reinforcing rejection spiral: prolonged GPS signal degradation increases innovation magnitude beyond the chi-squared gate; the gate then rejects valid fixes; state drift accumulates; subsequent fixes appear even more anomalous, causing further rejection. `robot_localization` EKF, lacking per-DOF chi-squared gating, is more permissive under sustained degradation and re-anchors when signal improves.

This is a known limitation of strict Mahalanobis gating. Adaptive gate relaxation (widening the threshold when the rejection rate over a sliding window exceeds a threshold) is the correct fix and is on the development roadmap.

D. Why RL-EKF Underperforms on Other Sequences

The NCLT GPS receiver reports covariances tighter than the actual sensor noise. `robot_localization` uses the NavSatFix covariance directly in its EKF fusion. FusionCore ignores the reported covariance and uses a user-configured baseline noise floor (σ_{xy} = 2.5 m for this benchmark). With better-calibrated innovation statistics, FusionCore's chi-squared gate passes more valid fixes and rejects fewer good measurements, the opposite of the November sequence failure.

This highlights a practical issue: GPS receivers routinely under-report covariance, and a filter that blindly trusts receiver-reported noise will over-reject valid measurements under the same Mahalanobis gate.

VIII. ROS 2 Integration

FusionCore is implemented as a ROS 2 lifecycle node. The `configure` transition initializes the filter from the YAML configuration; `activate` starts subscriptions and publishing. This enables orchestrated startup with Nav2's lifecycle manager.

A `/reset` service re-initializes the filter and clears the GPS reference origin without restarting the node, useful for multi-session mapping or GPS signal recovery.

FusionCore is available on the ROS binary apt repository for both Jazzy (Ubuntu 24.04) and Humble (Ubuntu 22.04):

```
sudo apt install ros-jazzy-fusioncore-ros
```

A Gazebo Harmonic simulation environment and example configurations for TurtleBot3, Clearpath Husky, and other common platforms are included in the repository. An official integration demo combining FusionCore with `rtabmap_ros` [13] was merged into the `rtabmap_ros` upstream repository [14], providing a reference configuration for visual SLAM plus FusionCore odometry fusion available to all `rtabmap_ros` users.

IX. Limitations and Future Work

The 2012-11-04 failure mode motivates the highest-priority future work: adaptive gate relaxation under sustained GPS degradation. Tracking the per-sensor rejection rate over a sliding window and widening the chi-squared threshold during prolonged rejection periods would recover the re-anchoring behavior seen in RL-EKF while preserving strict gating during normal operation.

Additional planned work includes: RTK fix-type-aware automatic noise scaling; radar Doppler ego-velocity fusion for platforms with 4D imaging radar; and a filter health diagnostic via innovation whiteness testing.

X. Conclusion

FusionCore is a 22-state UKF for ROS 2 that fuses IMU, wheel encoders, GPS, and Visual SLAM pose in a single lifecycle node at 100 Hz. Its design choices (continuous bias estimation, ECEF-native GPS, per-sensor chi-squared gating, adaptive noise, ZUPT, retrodiction-based delay compensation, and map-reinitialization recovery for VSLAM) address practical limitations of existing solutions. On five of six NCLT sequences, FusionCore reduces ATE RMSE by 2.0× to 12.9× versus `robot_localization` EKF. The source code, benchmark scripts, and reproducible configurations are publicly available at https://github.com/manankharwar/fusioncore.

Acknowledgments

The author thanks the open-source contributors to the FusionCore repository for their code contributions; Mathieu Labbe (matlabbe) for merging the FusionCore integration demo into `rtabmap_ros` and for endorsing the arXiv submission; and the NCLT dataset team at the University of Michigan for making their data publicly available.

References

[1] N. Carlevaris-Bianco, A. K. Ushani, and R. M. Eustice, "University of Michigan North Campus Long-Term Vision and Lidar Dataset," *The International Journal of Robotics Research*, vol. 35, no. 9, pp. 1023–1035, 2016.

[2] T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for the Robot Operating System," in *Proc. 13th Int. Conf. Intelligent Autonomous Systems (IAS-13)*, 2014.

[3] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Artech House, 2013.

[4] S. Lynen, M. W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, "A Robust and Modular Multi-Sensor Fusion Approach Applied to MAV Navigation," in *Proc. IEEE/RSJ IROS*, 2013.

[5] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart, "Robust Visual Inertial Odometry Using a Direct EKF-Based Approach," in *Proc. IEEE/RSJ IROS*, 2015.

[6] M. Madgwick, A. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG Orientation Using a Gradient Descent Algorithm," in *Proc. IEEE ICORR*, 2011.

[7] F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.

[8] S. Agarwal, K. Mierle, and T. C. S. Team, "Ceres Solver," http://ceres-solver.org, 2012.

[9] PROJ Contributors, "PROJ Coordinate Transformation Software Library," https://proj.org, 2023.

[10] M. Grupp, "evo: A Python Package for the Evaluation of Odometry and SLAM," https://github.com/MichaelGrupp/evo, 2017.

[11] T. Moore et al., "robot_localization Issue #780: UKF numerical stability," https://github.com/cra-ros-pkg/robot_localization/issues/780, 2022.

[12] T. Moore et al., "robot_localization Issue #951: UTM range error," https://github.com/cra-ros-pkg/robot_localization/issues/951, 2025.

[13] M. Labbe and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Incremental Mapping," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

[14] M. Kharwar and M. Labbe, "rtabmap_ros Pull Request #1419: FusionCore integration demo (TurtleBot3 Gazebo Harmonic)," https://github.com/introlab/rtabmap_ros/pull/1419, 2025.